A Comparative Analysis of Intrinsically Motivated Reinforcement Learning Algorithms in Robotics Manipulation

James Ogborne

BSc(Hons) Computer Science and Artificial Intelligence with Year long work placement

2025–2026

A Comparative Analysis of Intrinsically Motivated Reinforcement Learning Algorithms in Robotics Manipulation

submitted by James Ogborne

Declaration

This Dissertation is submitted to the University of Bath in accordance with the requirements of the degree of Bachelor of Science in the Department of Computer Science. No portion of the work in this Dissertation has been submitted in support of an application for any other degree or qualification of this or any other university or institution of learning. Except where specifically acknowledged, it is the work of the author.

1 Abstract

Sparse rewards represent one of the most persistent obstacles to deploying Deep Reinforcement Learning in complex robotic manipulation tasks, where an agent may execute thousands of actions before receiving any meaningful feedback. This study presents a comparative empirical evaluation of four Soft Actor-Critic (SAC) architectures on a robotic pick-and-place task, assessing the degree to which intrinsic motivation can mitigate this challenge and at what cost. Two classes of exploration augmentation are evaluated: novelty-based exploration via Random Network Distillation (RND) and prediction-based curiosity via the Intrinsic Curiosity Module (ICM), each benchmarked against fixed and automatically-tuned entropy coefficient baselines. Beyond conventional metrics of success rate and sample efficiency, this study introduces Mean Absolute Jerk as a primary evaluation criterion to assess the hardware viability of each converged policy.

The results reveal a fundamental tension that conventional benchmarking obscures. Among the evaluated architectures, RND proved well-matched to the task structure, achieving the highest success rate and fastest convergence, while ICM was undermined by the deterministic MuJoCo physics engine — whose predictable rigid-body kinematics caused its forward model to saturate, eliminating the exploration signal entirely. More critically, the Mean Absolute Jerk analysis exposes a systemic cost to intrinsic motivation that task metrics alone cannot capture: both intrinsically motivated agents exploit surplus post-task timesteps in fixed-length episodes to farm exploration bonuses through erratic, high-frequency joint actuation, producing trajectory profiles that would induce catastrophic mechanical strain on a physical manipulator. The fixed entropy baseline, despite its inferior task performance, produced the only policy compatible with zero-shot hardware deployment.

2 Acknowledgements

I would like to sincerely thank my supervisor, Ms Sophia Jones, for her invaluable guidance and support throughout this project. Her expertise and feedback were instrumental in shaping this work into what it is today.

Contents

3 Introduction

The ability to learn complex manipulation behaviours directly from environmental interaction, without requiring hand-crafted dynamic models, has made Deep Reinforcement Learning (DRL) an increasingly compelling framework for continuous control robotics [1, 2]. Unlike classical control methods, DRL agents can in principle acquire dexterous motor policies through experience alone. In practice, however, this promise is frequently undermined by the sparse reward problem: in tasks such as robotic pick-and-place, an agent may execute thousands of actions before the first successful task completion and the first reward signal. Without dense feedback to shape the policy gradient, learning is slow, unstable, or fails to converge entirely.

Two complementary strategies have emerged to address this. The first is architectural: the Soft Actor-Critic (SAC) framework [2] incorporates a maximum entropy objective that encourages broad exploratory behaviour implicitly, without requiring a hand-designed exploration strategy. The second is modular: auxiliary intrinsic motivation signals can be appended to the extrinsic reward, providing an agent-generated bonus that rewards the discovery of novel states or transitions independently of task success. This study focuses on two prominent instantiations of the latter approach—Random Network Distillation (RND) [3] and the Intrinsic Curiosity Module (ICM) [4]—which represent distinct approaches to generating intrinsic reward: state-space novelty and forward-dynamics prediction error, respectively.

While prior work has extensively benchmarked intrinsic motivation algorithms on simulated task performance, a critical dimension of evaluation is routinely overlooked: whether the policies produced are safe and viable for deploy-

ment on physical robotic hardware. A policy that achieves a high simulated success rate through erratic, high-frequency control signals may be entirely unsuitable for a real manipulator, where excessive jerk induces mechanical wear, excites structural resonances, and risks damaging both the robot and its payload. This gap between simulated optimality and physical viability is the central concern of this study.

3.1 Contributions

This study makes the following contributions:

- A controlled comparative evaluation of four SAC-based architectures—fixed entropy coefficient, automatically-tuned entropy coefficient, RND-augmented, and ICM-augmented—on the *Fetch-Pick-And-Place-v4* environment [5], with all architectural variables held constant to isolate the effect of each exploration mechanism.
- A mechanistic analysis of the exploration lifecycle of RND and ICM, demonstrating empirically that transition-based curiosity is structurally mismatched with deterministic physics simulators, and that this mismatch represents an extension of known ICM failure modes to the domain of deterministic rigid-body dynamics.
- The identification of an entropy-stiffness paradox: an inverse relationship between a policy's mathematical exploratory drive and its physical behaviour at the goal state, with implications for how entropy parameters manifest as actuator-level dynamics in manipulation tasks.
- The introduction of Mean Absolute Jerk as a primary evaluation criterion alongside task reward metrics, and the demonstration that intrinsic motivation induces hazardous reward farming behaviour in fixed-length episodes that produces policies physically unsafe for zero-shot hardware deployment.

4 Literature Review

4.1 Foundational Concepts in Robotic Reinforcement Learning

4.1.1 Markov Decision Processes

A Markov Decision Process (MDP) is characterised by $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$. $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, $\mathcal{T}$ a transition function $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \mathcal{P}(\mathcal{S})$ where $\mathcal{P}(\mathcal{S})$ is the probability distribution over the possible next states. Given the current state and action, a reward function $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to R$, and a discount factor $\gamma \in [0, 1)$. The agent interacts with the environment by selecting actions

to take from its policy $\pi : \mathcal{S} \to \mathcal{P}(\mathcal{A})$. The agent's goal is to maximise the expected cumulative discounted reward:

$$J(\pi) = E_{T\sim\pi} \sum_{t}^{\infty} \left[\gamma^t r(s_t, a_t)\right] \tag{1}$$

where $T = (s_0, a_0, s_1, a_1, \ldots)$ represents the trajectory of the agent induced by the policy π and the system dynamics $\mathcal{T}$.

4.1.2 Information Theory

To address the challenges of exploration, modern RL algorithms often draw upon concepts from information theory, particularly in defining the Maximum Entropy objective that underpins SAC.

Shannon Entropy H quantifies the amount of uncertainty or randomness inherent to a random variable. In the context of a discrete probability distribution P over a set of outcomes X, the Shannon Entropy is defined as:

$$H(P) = H(X) = -\sum_{x\in\mathcal{X}} P(x) \log P(x) \tag{2}$$

For a continuous policy $\pi(a|s)$ used in robotic control, the analogous measure is the differential entropy, which is defined as:

$$H(\pi(\cdot|s)) = -\int_{\mathcal{A}} \pi(a|s) \log \pi(a|s) da \tag{3}$$

4.1.3 Continuous Control and State of the Art Algorithms

In the space of continuous control, there are several very strong algorithms to consider. Particularly those that can handle high dimensional action spaces. These include, but are not limited to: Deep Deterministic Policy Gradient (DDPG) [1], Soft Actor-Critic (SAC) [2], and Proximal Policy Optimisation (PPO) [6]. DDPG is an off-policy actor-critic model that shows good performance when dealing with continuous spaces, but suffers from Q value overestimation. This typically leads to variants (TD3) [7] that use clipped double Q learning methods to use the minimum Q from an ensemble of functions. In addition to this, it is very sensitive to hyperparameter optimisation making it more difficult to train. A similar algorithm, SAC, which is also off-policy, adopts a maximum entropy framework which encourages exploration and improves robustness. The goal of this framework is to maximise the cumulative reward while ensuring the entropy of the policy is maximal. PPO, by contrast, is an on-policy method. It achieves stability by constraining the policy updates to avoid large, destructive shifts in behaviour. However, unlike off-policy algorithms, it cannot reuse past experience through replay buffers which tends to make it more sample inefficient despite being stable and easy to train. This replay buffer capability is particularly significant in sparse reward settings, where it enables techniques

such as Hindsight Experience Replay (HER) [8]. Rather than discarding failed episodes, HER retrospectively relabels them with the goals the agent actually achieved, effectively generating additional positive training signals from otherwise uninformative trajectories. This makes it a natural and powerful complement to off-policy algorithms like SAC in goal-conditioned manipulation tasks where extrinsic reward is rarely received. Overall, these algorithms are strong at learning a policy to control continuous systems. However, SAC may be the best suited for the current task. With its superior sample efficiency over PPO and its inherent exploration incentive (due to the maximum entropy term), it is positioned as a leading candidate for tackling sparse reward challenges within robotic manipulation.

4.1.4 Soft Actor-Critic (SAC)

SAC [2] is a model-free actor critic deep reinforcement learning algorithm that learns a stochastic policy. The core part of this algorithm is its entropy regularisation term $\alpha H(\pi)$. This term keeps the policy random and stops it converging too early to a suboptimal solution. This term is controlled by a temperature hyperparameter α which controls how influential the regularisation is. This can be kept constant or learnt alongside the policy. The objective function of SAC is defined as:

$$J(\pi) = \sum_t E_{(s_t,a_t)\sim\rho_\pi} \left[r(s_t, a_t) + \alpha H(\pi(\cdot|s_t)) \right] \tag{4}$$

Where $H(\pi(\cdot|s_t))$ is the entropy of the policy at state s_t and it is defined as:

$$-E_{a_t\sim\pi} \left[\log \pi(a_t|s_t) \right] \tag{5}$$

SAC is fundamentally made with two parts, the actor and the critic as shown in Figure 1. The actor is responsible for the policy $\pi_\theta(a|s)$, a network with parameters θ that outputs the probability distribution over the action space given the state. This actor is updated by maximising a soft objective that combines the expected Q-value with the entropy regularisation term, naturally resulting in stochastic, exploratory behaviour. One problem that can happen is the stochastic policy preventing backpropagation as random variables are not differentiable. The solution to this is through the reparametrisation trick [9]. Instead of sampling the action from the policy distribution, it is rewritten as a function of the distribution parameters with an added noise term.

$$a \sim \pi_\theta(a|s) \tag{6}$$

is changed to

$$a = \mu_\theta(s) + \sigma_\theta(s) \cdot \epsilon \tag{7}$$

It is assumed here that the policy distribution is a gaussian. $\epsilon \sim N(0, 1)$. The mapping from policy to action is now deterministic and differentiable allowing backpropagation to update the actor network. Instead of computing:

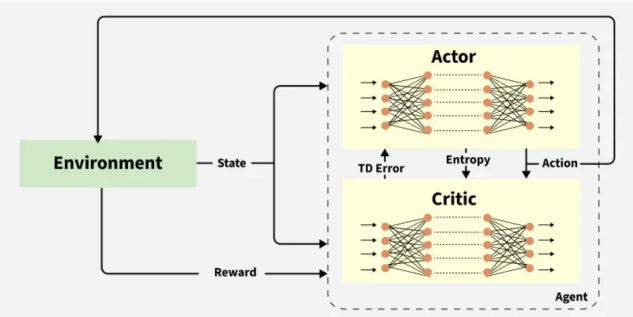

Figure 1: SAC Algorithm [10]

$$\nabla_\theta E_{a\sim\pi_\theta}\left[Q(s,a)\right] \tag{8}$$

You now compute:

$$\nabla_\theta Q(s, a(\epsilon)) \tag{9}$$

The role of the critic is to estimate the soft Q-value function representing the expected return and the entropy when taking an action in a state. Typically, there are 2 critic networks to mitigate Q value overestimation similar to TD3. These networks are updated by minimising the Bellman residual using targets that include this entropy term.

$$Q_\psi(s_t, a_t) = r(s_t, a_t) + \gamma E_{s_t+1\sim p}\left[V(s_{t+1})\right] \tag{10}$$

$$V(s_{t+1}) = E_{a_t+1\sim\pi}\left[Q_\theta(s_{t+1}, a_{t+1}) - \alpha log\pi(a_{t+1}, s_{t+1})\right] \tag{11}$$

Where ψ denotes the parameters of the target Q network and θ denotes the parameters of the current Q network.

The target network parameters ψ typically lag behind θ by using Polyak averaging like so:

$$\psi \leftarrow \tau\psi + (1-\tau)\theta \tag{12}$$

Where τ is a small constant ensuring the target lags behind the online network, but eventually catches up. This is a critical stabilisation technique allowing the target network to converge to a reliable value.

The SAC framework, particularly the entropy regularisation component $\alpha H(\pi)$, possesses an implicit form of intrinsic motivation. The temperature parameter α is a significant part of this project as it controls the agent's appetite for exploration. The ability to automatically tune this parameter means the agent can determine how much stochasticity is optimal [11]. This contrasts with other methods that require a separate auxiliary reward module, setting the stage for the comparative analysis in this study.

4.2 The Challenge of Exploration: Intrinsic Motivation

4.2.1 The Sparse Reward Problem

Classical RL algorithms typically operate within a dense reward environment, where the agent receives rewards or feedback on their actions very frequently. However, many real-world problems including robotic arm manipulation fall under the sparse reward environment category. By contrast, agents will receive feedback or rewards after a long sequence of successful actions. An example of such an environment is Montezuma's Revenge, a game infamous for its sparse rewards [12]. Robotics manipulation similarly provides a single final reward after the object has been transported to the correct position. When rewards are sparse, it becomes exceptionally difficult for an agent to assign credit to the actions that lead to success within huge trajectories. This difficulty results in two major performance failures:

- Sample inefficiency: The agent requires a vast number of actions in the environment to randomly stumble upon a successful trajectory.
- Failure to converge: In the worst case, the probability of success is too low, preventing the policy from receiving enough signal to ever converge to an optimal solution.

A solution to this issue is Intrinsic Motivation [13]. A way of providing dense reward signals to encourage meaningful exploration providing the necessary feedback where extrinsic rewards fail.

4.2.2 Taxonomy of Intrinsic Motivation (IM)

The majority of Intrinsic Motivation methods can be categorised into three broad areas. Prediction-based, novelty-based, and information-theoretic. Each formulation proposes a different proxy for utility in the absence of extrinsic reward.

Prediction-Based Methods Prediction-based methods operate on the intuition that a surprising event indicates a lack of knowledge from the agent. In its most general form, this involves a forward dynamics model that predicts the next state given the current state and action, with the intrinsic reward being proportional to the prediction error:

$$r_i \propto ||\hat{s_{t+1}} - s_{t+1}||^2 \tag{13}$$

Algorithms like ICM [4] build on this by computing the forward prediction error within a learnt feature space rather than the raw state space. However, these methods are prone to the noisy tv problem. In environments with stochastic transitions, an agent maximising prediction error will get consistently high rewards for observing these random events. This will cause the agent to "procrastinate" and not explore the environment or learn any useful skills.

Novelty-Based Methods Novelty-based methods focus on exploring states that are rarely visited. For discrete state spaces, this can be implemented through count-based methods [14], storing the frequency of visits at states and exploring the ones with the lowest visit count. Direct counting is impossible in high dimensional, continuous state spaces common in robotics. In continuous state spaces it is rare the agent will ever visit the same state twice, consequently every state visited appears "novel" rendering raw counts useless. To solve this methods like pseudo-counts [12] utilise density estimation to approximate state frequency. However, these generative models are computationally expensive to train alongside an RL agent and often struggle to scale to high resolution visual inputs. RND [3] addresses this limitation by using the the prediction error of a fixed random network as a proxy for novelty. Since the target is deterministic, it avoids the noisy tv problem.

Information-Theoretic Methods The information theoretic approach is grounded in Shannon's Information Theory, often focusing on mutual information or empowerment [15]. Empowerment methods define an intrinsic objective to maximise mutual information between the agent's actions and future states. Conceptually, the agent seeks states where it has the maximum "control" or influence over the environment. While theoretically robust, these methods face some hurdles.

- Computational Intractability: Calculating MI requires integrating over the entire state-action space which is computationally intractable for high dimensional continuous domains.
- Task Irrelevance: A high empowerment state is not necessarily a useful one. For example, a robot might maximise its empowerment by waving its arm in free space (where it has perfect control over its joints) rather than interacting with the object (where contact dynamics introduce uncertainty), leading to behaviour that actively avoids the task goal.

Algorithm Selection Given the computational intractability and task-avoidance tendencies of information-theoretic approaches in contact-rich environments, prediction-based and novelty-based methods present the most viable paths for exploration in continuous control. Consequently, this research selects **ICM** and **RND** as the primary algorithms for evaluation. Comparing these two state-of-the-art baselines allows for a direct, rigorous analysis of how forward-prediction and deterministic novelty perform against one another, particularly when faced with the complex contact dynamics of robotic simulation.

4.3 Detailed Analysis of Intrinsic Motivation Algorithms

4.3.1 Intrinsic Curiosity Module (ICM)

Intrinsic Curiosity Module (ICM) [4] is composed of three parts, the forward dynamics model (FDM), the inverse dynamics model (IDM), and the encoder.

The encoder converts high dimensional observations into a lower dimensional feature space containing only task-relevant features and agent-controllable information denoted as $\phi(s_t)$ for state s_t. The FDM predicts the next state in the feature space given the current state and an action:

$$\hat{\phi}(s_{t+1}) = f_\theta(\phi(s_t), a_t) \tag{14}$$

where f_θ is a neural network with parameters θ. The intrinsic reward signal is defined as the prediction error of the FDM:

$$r_t^i = \eta \cdot \frac{1}{2}\|\hat{\phi}(s_{t+1}) - \phi(s_{t+1})\|_2^2 \tag{15}$$

where η is a scaling factor for the curiosity reward. The corresponding forward loss is:

$$\mathcal{L}_{fwd} = \frac{1}{2}\|\hat{\phi}(s_{t+1}) - \phi(s_{t+1})\|_2^2 \tag{16}$$

The IDM predicts the action a_t that takes the agent from state $\phi(s_t)$ to $\phi(s_{t+1})$.

$$\hat{a_t} = g_\psi(\phi(s_t), \phi(s_{t+1})) \tag{17}$$

where g_ψ is a neural network with parameters ψ. It is trained using the mean squared error between predicted and executed actions:

$$\mathcal{L}_{idm} = \frac{1}{2}||a_t - \hat{a_t}||^2 \tag{18}$$

ICM jointly trains the encoder by combining the loss of the forward and inverse models:

$$\mathcal{L}_{ICM} = (1-\lambda)\mathcal{L}_{idm} + \lambda\mathcal{L}_{fwd} \tag{19}$$

where λ is a hyperparameter balancing the two losses. Importantly this loss backpropagates through the encoder shaping the feature space for curiosity-driven exploration.

Finally, the intrinsic reward r_t^i will be combined with the extrinsic reward from SAC:

$$r_{total} = r_t + r_t^i \tag{20}$$

One major drawback of ICM is that it only considers the influence of one-step actions in the reward. Because it measures novelty through immediate prediction error, ICM struggles in long-horizon tasks where the effects of actions are delayed, and in environments with stochastic and irreversible dynamics.

4.3.2 Random Network Distillation (RND)

To address the limitations of dynamics-based curiosity (like ICM), Burda et al. [3] proposed **Random Network Distillation (RND)**, a method that detects novelty without modelling the environment's transition dynamics. RND defines novelty through the prediction error of a fixed randomly initialised neural network.

Architecture and Objective The RND architecture consists of two neural networks that map an observation s_t to a feature vector $z \in R^k$:

1. **Target Network (f_θ):** A fixed network with randomly initialised parameters θ that are frozen (never updated). It produces the ground-truth feature vector $z_t = f_\theta(s_t)$.

2. **Predictor Network ($f_{\hat{\theta}}$):** A trainable network with parameters $\hat{\theta}$ that attempts to predict the output of the target network. It produces the prediction $\hat{z}_t = f_{\hat{\theta}}(s_t)$.

The intuition is that neural networks are good at generalising to similar states. If the agent visits a state s_t similar to those seen before, the Predictor Network will essentially "distil" the function of the Target Network, resulting in low error. Conversely, in novel unvisited states, the Predictor will fail to approximate the random mapping, resulting in high error.

The intrinsic reward r_t^i is defined as the Mean Squared Error (MSE) between the predictor and target outputs:

$$r_t^i = \|f_{\hat{\theta}}(s_t) - f_\theta(s_t)\|^2 \tag{21}$$

The Predictor Network is updated to minimise this same error via gradient descent:

$$\min_{\hat{\theta}} \mathcal{L}_{RND} = \|f_{\hat{\theta}}(s_t) - f_\theta(s_t)\|^2 \tag{22}$$

Comparison with ICM RND offers a distinct advantage over ICM regarding the **Noisy TV Problem**. Since the Target Network f_θ is deterministic and functions solely on the current state s_t (not the transition $s_t \to s_{t+1}$), the target values for stochastic transitions do not fluctuate. The Predictor Network can eventually learn to map even complex, noisy observations to their fixed random projections, causing the intrinsic reward to decay correctly as the state becomes "familiar," regardless of environmental stochasticity.

Limitations Despite its robustness, RND shares the **myopic limitation** of ICM. The reward signal is purely local—it indicates that the *current* state is novel but provides no information about whether this novelty contributes to the long-term goal. Furthermore, RND is highly sensitive to input scaling; without rigorous observation normalisation (e.g., clipping and running-mean filtering), the intrinsic reward can be dominated by input dimensions with large magnitudes rather than true novelty.

4.4 Robotic Manipulation and Simulation Environments

4.4.1 Pick and Place Task Formulations

To quantitatively assess the performance of the intrinsic motivation agents compared to the SAC baseline, this project adopts metrics standard in robotic reinforcement literature [5]. The primary objective is to transport an object to the goal position. An episode is considered successful if, at the final timestep, the euclidean distance between the object and the goal coordinates is within some distance tolerance ϵ.

$$S_{episode} = I(\|p_{obj} - g\|_2 < \epsilon) \tag{23}$$

Following the benchmarks [5] for the Fetch environment, we set $\epsilon = 0.05m$. The **Success Rate** is reported as the percentage of successful episodes over a rolling window of evaluations.

Given the focus on intrinsic motivation, sample efficiency is a critical metric. This is evaluated by measuring the number of environment interaction steps required for the agent to achieve a stable success rate (e.g., > 0.8). Agents that converge with fewer samples demonstrate superior exploration capabilities.

Beyond exploration efficiency, ensuring that learned policies are viable for physical deployment requires evaluation beyond task reward. High-frequency discontinuities in the control signal can cause abrupt changes between successive actions, inducing instability and unsafe intermediate states that make smoothness a critical property of any deployable policy [16]. In the context of reinforcement learning, this concern is particularly acute: policies optimised purely for simulated reward can exhibit erratic, high-frequency actuation that would be detrimental on physical hardware [17]. Smoothness of motion is formally quantified through the Mean Absolute Jerk — the average magnitude of the rate of change of acceleration across an episode:

$$J = \frac{1}{T-2} \sum_{t=2}^{T} \left| \frac{\Delta^2 \mathbf{v}_t}{\Delta t^3} \right|_2 \tag{24}$$

where $\mathbf{v}_t \in R^3$ is the Cartesian velocity vector at timestep t and Δt is the environment timestep. In this study, jerk is approximated in task-space via the Cartesian velocity observations provided by the environment, as the standard wrapper abstracts direct joint-level actuator states. A lower Mean Absolute Jerk indicates smoother, more mechanically conservative motion. This study treats Mean Absolute Jerk as a co-equal evaluation criterion alongside task performance metrics, on the basis that a policy which is unsafe for physical hardware cannot be considered truly optimal regardless of its simulated success rate.

4.4.2 The Role of Simulation: Benchmarking in MuJoCo

Historically, the majority of Intrinsic Motivation research [4, 3] has been benchmarked in environments like Atari and MuJoCo [5]. This project utilises **MuJoCo** (Multi-Joint dynamics with Contact), a highly optimised physics engine tailored for continuous control tasks. While recent high-fidelity simulators offer photorealistic rendering, MuJoCo was selected as the primary environment for this project for two critical reasons:

1. **Computational Efficiency:** Intrinsic motivation algorithms, particularly during the exploration phase, are highly sample-inefficient. MuJoCo's exceptionally fast, CPU-based simulation speeds allow for the rapid collection of millions of environment steps, enabling comprehensive training and hyperparameter tuning that would be computationally prohibitive in ray-traced simulators.

2. **Algorithmic Isolation:** By utilising MuJoCo's clean state observations, the evaluation directly isolates the fundamental performance differences between exploration strategies like ICM and RND. This prevents the algorithms' intrinsic motivation capabilities from being confounded by the massive representation-learning overhead required to process photorealistic visual inputs.

4.5 Synthesis and Proposed Research

4.5.1 The Entropy vs. Curiosity Trade-off

The literature survey reveals a tension between two dominant paradigms of exploration in continuous control: the **implicit** exploration of Maximum Entropy RL (SAC) and the **explicit** exploration of Intrinsic Motivation (ICM/RND).

While SAC's entropy term $\alpha H(\pi)$ effectively prevents policy collapse and encourages local stochasticity, will it be insufficient for tasks with "bottleneck" states—specific configurations (like grasping an object) that must be reached to unlock further exploration? If this is the case, explicit methods like ICM could provide directed exploration signals that guide the agent through these bottlenecks.

However, combining these methods raises the **Redundancy Hypothesis**: Does adding an intrinsic reward r^i to a Maximum Entropy objective result in synergistic exploration, or does it lead to over-exploration where the agent ignores the extrinsic task? This project aims to empirically resolve these ambiguities through a comparative ablation study.

4.5.2 Research Questions

Based on the identified gaps in the literature, this project poses the following core research questions:

- **RQ1 (The Baseline Question):** Is the native entropy regularisation in SAC sufficient to solve the sparse-reward fetch pick-and-place task, or is an auxiliary intrinsic reward signal required for convergence?
- **RQ2 (The Mechanism Question):** Which intrinsic motivation formulation yields higher sample efficiency and final success rates: the prediction-error maximisation of **ICM**, or the random network distillation of **RND**?
- **RQ3 (The Robustness Question):** How do these algorithms compare in terms of stability (variance across seeds) and motion quality (jerk)?

4.5.3 Proposed Methodology

To answer these questions, the project has implemented four distinct agent configurations within the MuJoCo environment:

1. **Baseline Fixed Entropy:** Standard SAC with fixed temperature α (Implicit Exploration).

2. **Baseline Tuned Entropy:** Standard SAC with learnable temperature α (Learnt Implicit Exploration).

3. **SAC + ICM:** Augmenting the reward with Inverse Dynamics prediction error (Explicit Prediction).

4. **SAC + RND:** Augmenting the reward with Distillation error (Explicit Novelty).

These agents were evaluated on the Success Rate, Sample Efficiency (Time to Convergence), and Trajectory Smoothness metrics.

4.5.4 Synthesis

The sparse reward problem in robotic manipulation remains a significant barrier to the widespread adoption of Reinforcement Learning in industry. While Soft Actor-Critic provides a robust foundation for continuous control, its reliance on undirected entropy maximisation may falter in complex, multi-stage manipulation tasks. Intrinsic Motivation offers a promising solution by endowing agents with an internal drive to explore. By systematically comparing the Entropy-based, Prediction-based, and Novelty-based paradigms, this project seeks to identify the optimal exploration strategy for robotic pick-and-place tasks, contributing to the broader effort of making robotic learning more sample-efficient and autonomous.

5 Methodology

5.1 High-Level Overview

Figure 2 illustrates the interaction between the proposed agent architecture and the environment. At each time step t, the agent observes the current state s_t and the SAC policy network outputs an action a_t. The environment executes this action, transitioning to the next state s_{t+1} and yielding an extrinsic reward r_e.

Concurrently, the intrinsic motivation module processes the transition tuple (s_t, a_t, s_{t+1}) to compute an intrinsic reward, r_i, based on the agent's novelty or prediction error. This intrinsic signal is combined with the extrinsic reward to formulate the total augmented reward:

$$r_t = \beta r_i + r_e \tag{25}$$

where β serves as the intrinsic reward scaling factor. The SAC agent then utilises this augmented reward sequence (s_t, a_t, r_t, s_{t+1}) to update its actor and critic networks, guiding the policy toward more robust exploration and task completion.

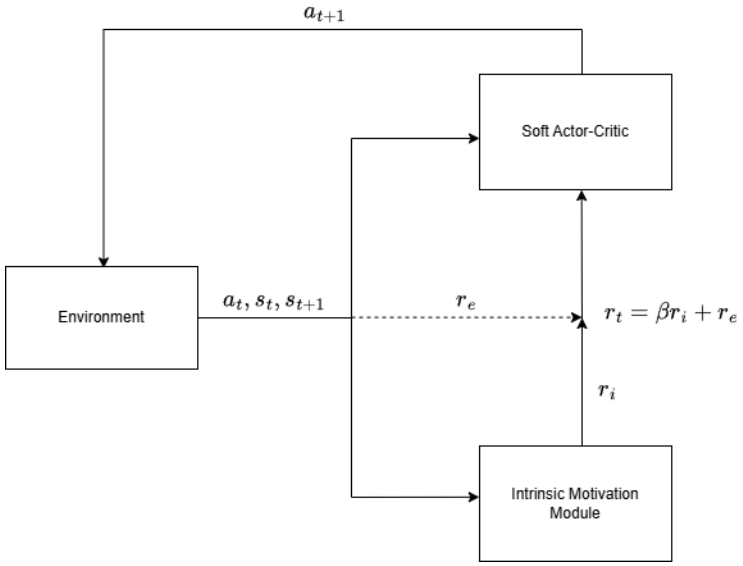

Figure 2: High-level architecture of the reinforcement learning lifecycle, illustrating the integration of Soft Actor-Critic (SAC) with an Intrinsic Motivation module.

5.2 Soft Actor-Critic (SAC) Architecture

The base algorithm for the control agent is Soft Actor-Critic (SAC) [2]. SAC is an off-policy, actor-critic deep reinforcement learning algorithm based on the maximum entropy framework. In this framework, the actor aims to maximise a trade-off between the expected return and the entropy of the policy. This encourages robust exploration and prevents the policy from prematurely converging to local optima.

The standard objective is augmented with an entropy term $\mathcal{H}$, scaled by a temperature parameter α that determines the relative importance of the entropy term against the reward:

$$\pi^* = \arg\max_{\pi} \sum_{t=0}^{T} E_{(s_t,a_t)\sim\rho_\pi} \left[r_t + \alpha \mathcal{H}(\pi(\cdot|s_t)) \right] \tag{26}$$

where r_t is the extrinsic reward, and ρ_π represents the state-action marginals of the trajectory distribution induced by policy π. To optimise this objective, SAC concurrently learns a policy network (the actor) and two soft Q-networks (the critics).

5.2.1 Critic Network Update

To mitigate the overestimation bias common in standard Q-learning, SAC employs clipped double Q-learning [7]. Two independently initialised Q-networks, parameterised by θ_1 and θ_2, are trained to minimise the soft Bellman residual. The loss function for each critic $i \in \{1, 2\}$ is defined as the Mean Squared Error (MSE):

$$J_Q(\theta_i) = E_{(s_t,a_t)\sim\mathcal{D}}\left[\frac{1}{2}\left(Q_{\theta_i}(s_t, a_t) - y_t\right)^2\right] \tag{27}$$

where $\mathcal{D}$ is the augmented replay buffer, and the target y_t is calculated using the target Q-networks and the next action sampled from the current policy:

$$y_t = r_t + \gamma(1 - d_t)\left(\min_{j=1,2} Q_{target,j}(s_{t+1}, a_{t+1}) - \alpha \log \pi_\phi(a_{t+1}|s_{t+1})\right) \tag{28}$$

where $d_t \in \{0, 1\}$ represents the terminal state flag, ensuring future returns are truncated at the end of an episode.

Because the robotic manipulation tasks require bounded continuous actions, the action sampled from the actor's Gaussian distribution is squashed using a hyperbolic tangent function (tanh). To account for this non-linear transformation, the log probability $\log \pi_\phi$ is dynamically corrected using the Jacobian determinant of the tanh function.

During optimisation, the gradients for both critic networks are computed simultaneously. To ensure training stability and prevent exploding gradients—a common issue when integrating non-stationary intrinsic rewards—a global gradient clipping threshold of 1.0 (via L_2 norm) is applied to the critic parameters prior to the optimiser step. Finally, the target networks are updated via exponential moving average (polyak averaging) to ensure stable target values.

5.2.2 Actor Network Update

For continuous control in robotics manipulation, action sample distribution is modelled as a Gaussian distribution. To allow gradients to backpropagate through the stochastic sampling process, the reparametrisation trick is utilised. An action is sampled according to $a_t = f_\phi(\epsilon_t; s_t)$, where $\epsilon_t \sim \mathcal{N}(0, I)$ is an input noise vector, and the output is bounded by a hyperbolic tangent function to satisfy the action limits.

The actor network, parametrised by ϕ, is updated by minimising the expected KL-divergence between the policy and the exponential of the soft Q-function:

$$J_\pi(\phi) = E_{s_t\sim\mathcal{D},\epsilon_t\sim\mathcal{N}}\left[\alpha \log \pi_\phi(f_\phi(\epsilon_t; s_t)|s_t) - \min_{i=1,2} Q_{\theta_i}(s_t, f_\phi(\epsilon_t; s_t))\right] \tag{29}$$

5.2.3 Automatic Entropy Tuning

Because the scale of the reward can vary significantly—especially when integrating non-stationary intrinsic rewards—selecting a fixed temperature parameter

α is often sub-optimal. Therefore, this study evaluates both a baseline SAC with a fixed α and a variant where α is treated as a dual variable.

In the tuned configuration, α is automatically adjusted during training to match a target entropy heuristic, $\bar{\mathcal{H}}$, formulated as a constrained optimisation problem. The loss function for the temperature parameter is:

$$J(\alpha) = E_{s_t \sim \mathcal{D}, a_t \sim \pi_\phi} \left[-\alpha \log \pi_\phi(a_t|s_t) - \alpha \bar{\mathcal{H}} \right] \tag{30}$$

For the robotic manipulation tasks in this environment, the target entropy is set to the standard heuristic of $\bar{\mathcal{H}} = -\dim(\mathcal{A})$ [11], ensuring the agent maintains a minimum level of stochasticity throughout the learning process.

5.3 Hindsight Experience Replay (HER)

Robotic manipulation tasks are frequently formulated as goal-conditioned reinforcement learning problems. In this framework, the agent is tasked not only with maximising a general reward but with achieving a specific desired goal state, $g \in \mathcal{G}$.

A primary challenge in goal-conditioned manipulation is that the extrinsic reward, r_e, is typically sparse and binary. For instance, the environment yields a reward of 0.0 if the goal is achieved within a certain tolerance, and a negative penalty -1.0 otherwise. Under a standard uniform sampling strategy, the replay buffer becomes overwhelmingly populated with failed trajectories. Consequently, the Soft Actor-Critic (SAC) networks receive virtually no positive reinforcement during early training, causing the Q-value gradients to vanish before the policy can discover how to solve the task.

To overcome this sparse reward problem, Hindsight Experience Replay (HER) [8] is integrated into the replay buffer architecture. While the original formulation of HER introduces several goal-sampling strategies - namely final, future, episode, and random - this implementation utilises the 'future' strategy, as it has been empirically shown to provide the most significant performance gains in multi-goal environments. When an episode ends, the agent may not have reached its intended goal g, but it has nevertheless achieved some physical state. HER retrospectively assumes that this achieved state was the intended goal all along.

In this implementation, the replay buffer operates by processing full episodes and employing a "future" relabelling strategy. For every standard transition in a trajectory, $\tau = (s_t, g, a_t, r_e, s_{t+1})$, the original transition is stored in the buffer. Subsequently, the algorithm randomly samples up to $k = 4$ future achieved states from the remainder of that same trajectory.

For each sampled future state, a synthetic transition is generated. The original desired goal g is replaced with the hindsight goal g' (the achieved state from the future time step). Because the agent is now guaranteed to have reached this new goal g' at that future point in the trajectory, the reward for this synthetic transition is re-evaluated and overridden to denote success ($r'_e = 0.0$).

This yields the augmented transition tuple:

$$\tau' = (s_t, g', a_t, r'_e, s_{t+1}) \tag{31}$$

By artificially injecting these successful transitions into the training batches, HER effectively converts every trajectory into a successful demonstration of how to reach *some* state within the environment. During optimisation, mini-batches are uniformly sampled from this augmented buffer. The SAC critic networks utilise these relabelled transitions to learn a generalised, goal-conditioned Q-function, $Q_\theta([s_t \parallel g], a_t)$.

When combined with intrinsic motivation, HER addresses the sparse extrinsic reward by providing dense retrospective successes, while the intrinsic modules address the exploration problem by incentivising the agent to discover a wider variety of these achievable states.

5.4 Intrinsic Motivation Formulations

To overcome the exploration challenges inherent in complex robotic manipulation, the baseline SAC agent is augmented with intrinsic motivation. This provides the agent with dense exploratory rewards, even in the absence of external environmental feedback. Two distinct intrinsic motivation formulations are evaluated: Random Network Distillation (RND) [3] and a modified Intrinsic Curiosity Module (ICM) [4].

5.4.1 Random Network Distillation (RND)

RND incentivises exploration by rewarding the agent for visiting novel states. The architecture consists of two neural networks: a fixed, randomly initialised target network, $f(s)$, and a predictor network, $\hat{f}(s;\theta)$.

The target network maps the observation to a latent representation and its weights remain frozen throughout training. The predictor network is trained to mimic the target network by minimising the Mean Squared Error (MSE) between their outputs for a given state observation s_{t+1}:

$$\mathcal{L}_{RND} = E_{s_{t+1}\sim\mathcal{D}}\left[||\hat{f}(s_{t+1};\theta) - f(s_{t+1})||_2^2\right] \tag{32}$$

As the agent visits a state repeatedly, the predictor network learns to accurately map the state to the target's output, reducing the prediction error. Conversely, unseen or novel states yield a high prediction error. This error is utilised as the intrinsic reward:

$$r_i = ||\hat{f}(s_{t+1};\theta) - f(s_{t+1})||_2^2 \tag{33}$$

The implementation of this adapts the architecture from the CleanRL library [18], modifying it for compatibility with the continuous action space and SAC training loop used in this study.

5.4.2 Modified Intrinsic Curiosity Module (ICM)

The standard ICM generates intrinsic rewards based on the agent's inability to predict the consequences of its own actions. Typically, this involves an inverse dynamics model to learn a feature representation that is invariant to uncontrollable environmental noise. However, because the robotic manipulation task in this environment utilises a "pure", low-dimensional kinematic state space devoid of pixel-based noise, feature encoding is redundant.

Consequently, the ICM architecture utilised in this study is streamlined to rely solely on a forward dynamics model, operating directly on the raw state space. This reduces computational overhead while preserving the core curiosity mechanism.

The forward model, parametrised by ψ, takes the current state s_t and the executed action a_t as inputs to predict the next state, $\hat{s}_{t+1}$. The network is trained by minimising the prediction error against the actual next state s_{t+1} experienced in the environment:

$$\mathcal{L}_{ICM} = E_{(s_t,a_t,s_{t+1})\sim\mathcal{D}}\left[||\hat{s}_{t+1}(s_t, a_t; \psi) - s_{t+1}||_2^2\right] \tag{34}$$

The intrinsic reward is then formulated as the MSE of this forward prediction, effectively rewarding the agent for taking actions that lead to unpredictable state transitions:

$$r_i = ||\hat{s}_{t+1}(s_t, a_t; \psi) - s_{t+1}||_2^2 \tag{35}$$

5.5 Reward Integration and Scaling

A significant challenge in combining intrinsic and extrinsic motivation is the inherent non-stationarity and differing magnitudes of the reward signals. The extrinsic reward, r_e, provided by the environment typically operates on a consistent, predefined scale. In contrast, intrinsic rewards generated by prediction-error formulations, such as RND and ICM, are highly non-stationary. As the agent's predictive models improve over the course of training, the prediction errors decrease, causing the raw intrinsic reward to naturally decay. Without intervention, this decay causes the intrinsic signal to become negligible relative to the extrinsic reward, thereby prematurely halting exploration.

To ensure the intrinsic reward maintains a meaningful and consistent influence on the policy updates throughout the training lifecycle, the raw intrinsic signal is dynamically scaled using a running standard deviation. Let $r_{i,t}^{raw}$ denote the raw intrinsic reward computed at time step t. The scaled intrinsic reward, $r_{i,t}$, is calculated as:

$$r_{i,t} = \frac{r_{i,t}^{raw}}{\sigma_{i,t} + \epsilon} \tag{36}$$

where $\sigma_{i,t}$ is the running standard deviation of all intrinsic rewards observed up to time step t, and ϵ is a small constant (1×10^{-4}) added for numerical stability to prevent division by zero.

This running normaliser ensures that the intrinsic reward maintains a consistent variance, regardless of how accurate the underlying prediction models become. Once scaled, the intrinsic reward is combined with the extrinsic reward to formulate the augmented reward signal used to train the Soft Actor-Critic agent:

$$r_t = r_{e,t} + \beta r_{i,t} \tag{37}$$

where β is a fixed hyperparameter that dictates the relative importance of the exploratory intrinsic drive against the task-oriented extrinsic reward.

5.6 Data Processing and State Normalisation

Deep reinforcement learning architectures exhibit varying degrees of sensitivity to the scale and distribution of input features. Raw state observations from robotic manipulation environments often encompass heterogeneous units, leading to varying orders of magnitude across the state vector.

Through empirical evaluation, it was observed that applying a global running normalisation to the state observations prior to processing them through the Soft Actor-Critic (SAC) networks resulted in performance degradation. Because running statistics continuously update throughout training, normalising the actor and critic inputs effectively renders the observation space non-stationary. This shifting distribution destabilised the SAC policy updates. Consequently, the raw, unscaled state vector s_t is passed directly to the SAC agent to preserve the stationarity of the Markov Decision Process (MDP).

Conversely, the intrinsic motivation modules (RND and ICM) exhibited extreme sensitivity to unscaled inputs. In RND, feeding unnormalised states into the fixed, randomly initialised target network often leads to saturated activations or prediction errors dominated by state features with the largest magnitudes. To ensure stable optimisation and scale-invariant prediction errors, the state observation pipeline is bifurcated. While SAC receives raw states, the inputs directed to the RND and ICM networks are dynamically scaled using a running statistical tracker.

To efficiently compute these running statistics across training batches without retaining the entire state history in memory, Welford's online algorithm [19] is employed. For a given mini-batch of size m with its own empirical mean μ_B and variance σ_B^2, the global running mean μ_t and variance σ_t^2 are updated incrementally. Let N be the total number of states observed prior to the current batch. The updated moments are computed as:

$$\Delta = \mu_B - \mu_t \tag{38}$$

$$\mu_{t+1} = \mu_t + \Delta \frac{m}{N+m} \tag{39}$$

$$\sigma_{t+1}^2 = \frac{N\sigma_t^2 + m\sigma_B^2 + \Delta^2 \frac{N \cdot m}{N+m}}{N+m} \tag{40}$$

Once the running statistics are updated, the state vector is normalised. Let $s_t^{(k)}$ denote the k-th feature of the raw state vector at time step t. The normalised feature, $\tilde{s}_t^{(k)}$, exclusively utilised by the intrinsic modules, is computed as:

$$\tilde{s}_t^{(k)} = clip\left(\frac{s_t^{(k)} - \mu_{t+1}^{(k)}}{\sqrt{\sigma_{t+1}^{(k)2} + \epsilon}},\ -c,\ c\right) \tag{41}$$

where $\epsilon = 1 \times 10^{-4}$ is a constant added to prevent division by zero, and the normalised value is bounded by a clipping threshold, $c = 5.0$, to prevent extreme outliers from causing destabilising spikes in the network activations.

This dual-pipeline approach ensures that the base control policy benefits from a stationary environment, while the curiosity modules operate on the standardised distributions required for stable representation learning.

5.7 Network Architectures and Hyperparameters

This final section details the exact neural network architectures and hyperparameter configurations used to implement the Soft Actor-Critic (SAC) baseline and the intrinsic motivation modules shown in tables 3 and 4 in the appendix.

All networks were trained using the Adam optimiser. To ensure numerical stability, the actor network bounds its log standard deviation output within the range $[-20, 2]$. Furthermore, specific initialisation strategies were critical for the Random Network Distillation (RND) module: both networks utilise Orthogonal initialisation [20] to promote efficient gradient flow during learning.

6 Experiment Design

6.1 Experimental Environment: Fetch-Pick-And-Place-v4

The study utilises the *Gymnasium Fetch-Pick-And-Place-v4* [5] environment, a standard benchmark for robotic manipulation. The setup comprises a 7-Degree-of-Freedom (7-DOF) Fetch research arm situated before a workspace containing a table, a moveable block, and a red target designated as the goal state.

The task is characterised by a **sparse reward structure**, where the agent receives a reward of $r = 0$ only when the block is within a specified proximity of the target, and $r = -1$ for all other timesteps. This binary feedback creates a "plateau" in the reward landscape, making it difficult for standard RL agents to learn without significant exploration.

- **Action Space:** A 4D continuous space representing the (x, y, z) displacement of the end-effector and a 1D scalar for the gripper's finger actuation.
- **Observation Space:** A 25D goal-aware vector. This includes the kinematic state of the robot joints, the Cartesian coordinates and linear velocity of the block, and the relative distance to the target.

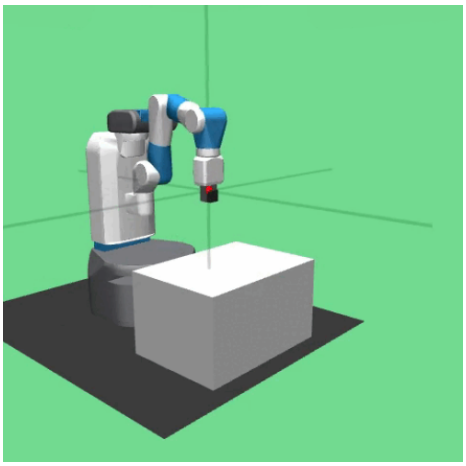

Figure 3: Fetch pick and place environment

- **Rewards:** The rewards can be initialised as sparse or dense. Sparse will return 2 possible values; $r = 0$ if the block has reached the target position, or $r = -1$ for anything else. The block is considered to have reached the goal if the distance between itself and the target is lower than $0.05m$. Dense, on the other hand, returns a reward that is the negative euclidean distance between the achieved goal position and the desired goal. For the following experiment, sparse reward setting will be used.

- **Transition Dynamics:** The environment transitions deterministically, where the next state is a fixed function of the current state and action.

$$s_{t+1} = f(s_t, a) \tag{42}$$

  These dynamics are governed by the MuJoCo physics engine [5]. While the transition probability is deterministic from a simulation standpoint, the underlying rigid-body dynamics (contact forces, frictional constraints, gravity) present a highly non-linear mapping for the agent to learn.

6.2 Comparative Framework

To isolate the impact of Intrinsic Motivation (IM), this study adopts a comparative approach using Soft Actor-Critic (SAC) combined with Hindsight Experience Replay (HER) as the foundational architecture. Four distinct agent configurations were evaluated:

1. **Baseline (Fixed):** SAC + HER with a static temperature parameter (α), representing a standard maximum-entropy RL approach.

2. **Baseline (Learnt):** SAC + HER with an automated entropy adjustment (learnt α), allowing for dynamic exploration.

3. **ICM-Augmented:** The baseline augmented with the *Intrinsic Curiosity Module*, which provides a reward based on the agent's inability to predict the consequences of its actions.

4. **RND-Augmented:** The baseline augmented with *Random Network Distillation*, which rewards the agent for state novelty by measuring the prediction error between a trained network and a fixed, randomly initialised target network.

To ensure statistical validity and mitigate the effects of random initialisation, the performance of each algorithm is averaged across four independent agent runs of length 4.75 million timesteps. Furthermore, all agents share identical hyperparameters—including learning rate, batch size, and neural network depth—ensuring that performance variances are attributable solely to the auxiliary reward modules.

6.3 Evaluation Metrics

The following metrics are selected to provide a holistic view of the agent's performance, focusing on both objective task completion and the real-world feasibility of the learned policies.

6.3.1 Functional Performance

- **Success Rate (SR):** Defined as the percentage of episodes where the block is successfully placed within the target radius. This is the primary measure of the agent's task-solving capability.

- **Mean Steps to Convergence:** This measures *sample efficiency* by calculating the number of environment interactions required for the agent to maintain a stable success rate (e.g., > 0.8).

6.3.2 Hardware Viability: Jerk Analysis

Mean Absolute Jerk is recorded at incremental checkpoints throughout training to track the evolution of trajectory smoothness across the learning lifecycle. For each checkpoint, Mean Absolute Jerk is computed over a fixed set of evaluation episodes using the discrete approximation introduced previously, applied to the Cartesian velocity observations, indices 20-22 (inclusive), extracted from the 25D observation vector, with an environment timestep of $\Delta t = 0.04$s, corresponding to the environment's 25Hz simulation frequency. The resulting Mean Absolute Jerk values, expressed in m/s^3, provide a quantitative measure of the deployment-readiness of each policy at each stage of training.

6.3.3 Mechanistic Interpretability

Intrinsic Reward Magnitude: Tracking the auxiliary reward signal over time reveals the "exploration lifecycle". This signal should naturally decay as the environment becomes familiar and the agent's uncertainty decreases.

7 Results

7.1 Functional Performance and Sample Efficiency

The following section presents the empirical evaluation of all four architectures across three sequential dimensions of analysis. First, task performance and sample efficiency are assessed through success rate and mean reward metrics, establishing a functional hierarchy across algorithms. Second, the intrinsic reward dynamics of the RND and ICM agents are examined to provide mechanistic insight into why their exploration profiles diverge. Third, and most critically, the hardware viability of each converged policy is evaluated through Mean Absolute Jerk measurements recorded at incremental stages of training, alongside qualitative observations of agent behaviour. Together, these three layers of analysis move beyond conventional simulated benchmarking to assess not only how well each agent learns, but whether the resulting policies are safe and viable for deployment on physical robotic hardware.

Algorithm	**Success Rate**	**Convergence Timesteps**
SAC+HER (Fixed α)	0.81	4,109,950
SAC+HER (Learnt α)	0.91	1,581,350
RND Augmented	0.93	1,326,200
ICM Augmented	0.90	2,093,200

Table 1: Algorithm success rate and mean steps to convergence (> 0.8 Success Rate)

Algorithm	**Mean Reward per Episode**
SAC+HER (Fixed α)	-31.99
SAC+HER (Learnt α)	-14.67
RND Augmented	-14.63
ICM Augmented	-14.91

Table 2: Mean reward per episode for each algorithm

7.2 The Role of Entropy Scheduling

Table 1 shows that the baseline architecture with a learnt entropy coefficient is superior to the fixed coefficient. It achieves a 10% Success Rate increase and

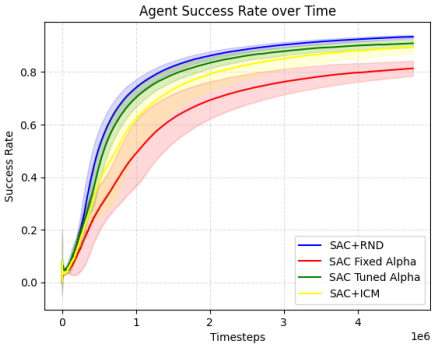

Figure 4: Agent Success Rate Over Training Timesteps

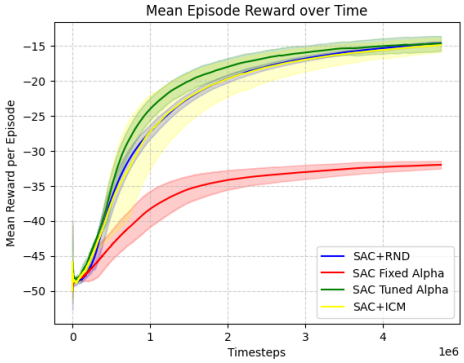

Figure 5: Mean Reward per Episode

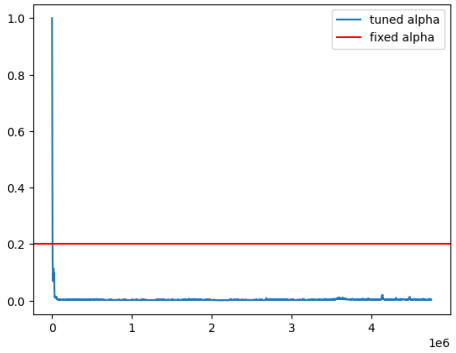

Figure 6: Entropy Coefficient (α) Over Time

reaches a success rate of 0.80 over 2.5 million time steps ahead of the fixed version.

Figure 6 demonstrates how both sets of entropy coefficients vary over time. The fixed temperature was maintained at 0.2 throughout, while the learnt temperature exhibited an initial steep decay from 1.0 down to 0.09, eventually plateauing to 0.0035 around timestep 26,000. It becomes clear that reduced policy entropy is vital for securing a higher success rate. The persistent randomness dictated by the fixed α transitions from a useful exploration tool into a strict hindrance during training.

This hindrance stems from the fact that robotic manipulation—specifically the Fetch pick-and-place task—requires a high degree of precision and fine-motor dexterity to satisfy the strict 0.05m success radius. The persistent entropy mandated by the fixed α architecture directly limits this precision by preventing the agent's action distribution from collapsing into a highly deterministic, confident state. Consequently, the fixed α agent suffers from two primary modes of inac-

curacy. First, when navigating to the target, the persistent entropy manifests as spatial indecision; rather than executing a direct, highly accurate trajectory, the agent exhibits smooth but lethargic approximations, frequently ending the episode before perfectly centring the block. Second, when attempting to stabilise the block in mid-air, the mandated randomness prevents the agent from applying the rigid, precise micro-corrections necessary to keep the block strictly within the goal bounds. It settles for being "close enough," which satisfies its entropy bonus but ultimately registers as a task failure.

Table 2 and figure 5 illustrate that the learnt α architecture achieves a substantially higher mean reward per episode (-14.67) compared to the fixed α baseline (-31.99). This halving of the negative accumulated reward might initially seem disproportionate given the modest 10% difference in overall Success Rate. However, this discrepancy is directly explained by the environment's sparse reward structure. Because the agent receives a step penalty of $r = -1$ until the target is reached, the cumulative episode reward acts as a precise proxy for execution speed and trajectory efficiency. Qualitative observations confirm that the learnt α agent transports the block to the target in significantly fewer timesteps. This further corroborates the earlier observation: the persistent entropy in the fixed α policy induces lethargic, imprecise trajectories, whereas the decaying entropy of the tuned policy facilitates the swift, decisive, and highly accurate manipulation required for optimal physical execution.

7.3 Interpretation of Intrinsic Motivation

As demonstrated in Tables 1 and 2, the integration of Random Network Distillation (RND) yielded the most performant policy among the evaluated architectures. The RND-augmented agent achieved a peak Success Rate of 0.93 and reached the convergence threshold in just 1.3 million timesteps. This represents a significant enhancement in sample efficiency, converging approximately 200,000 timesteps faster than the best-performing baseline (Learnt α). Furthermore, the RND agent secured the highest mean reward per episode (-14.63), indicating that the auxiliary exploration signal did not distract the agent from the primary objective; rather, it facilitated highly direct and efficient trajectories to the target state.

Conversely, the Intrinsic Curiosity Module (ICM) presented a highly contrasting performance profile. While it achieved a competitive final Success Rate of 0.90, it required 2,093,200 timesteps to reach convergence. This makes it significantly less sample-efficient than both the RND agent and the non-augmented Learnt α baseline.

To understand the mechanistic drivers behind this functional disparity, we must examine the aggregate exploration lifecycles of both the RND and ICM agents.

Figure 7 plots the magnitude of the intrinsic reward signals generated by the exploration modules over the training duration. Note that the plotted values represent the normalised signal scaled via a running standard deviation, as detailed in the methodology, rather than the raw prediction error.

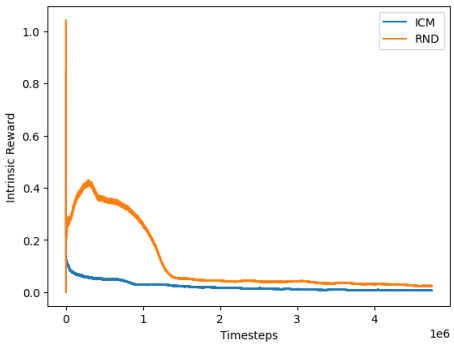

Figure 7: Normalised intrinsic reward over time for RND and ICM augmented agents

The plot reveals two fundamentally different exploration profiles. The RND agent exhibits a massive spike in intrinsic reward during the initial stages of training, followed by a steady decay. Because RND calculates its reward based on the prediction error of a fixed random network, it acts as a direct proxy for spatial state novelty. The 25D goal-aware observation space generates large initial prediction errors, driving the agent to rapidly interact with the environment and discover the sparse reward state. As the agent maps the Cartesian space, the prediction error naturally anneals, allowing a smooth transition into policy exploitation.

In stark contrast, the ICM intrinsic reward signal lacks this initial novelty spike. Instead, it begins at a comparatively low magnitude and decays steadily, remaining consistently lower than the RND signal throughout the training lifecycle. This discrepancy is a direct result of ICM's underlying architecture. Unlike RND, which measures state novelty, ICM generates intrinsic reward by measuring the error in predicting the forward dynamics of the environment (i.e., predicting s_{t+1} given s_t and a_t).

Because the MuJoCo physics engine driving the Fetch environment is fundamentally deterministic, the transition dynamics of the robotic arm are mathematically consistent. The ICM's forward model learns to predict these deterministic rigid-body kinematics very quickly. Consequently, moving the arm to a new, unvisited area of the workspace does not generate a high intrinsic reward because the *transition* to get there was perfectly predictable, even if the destination state was novel.

Because the ICM signal remained persistently low, it failed to provide the necessary exploration "kick" required to rapidly overcome the sparse reward plateau ($r = 0$). Without a strong intrinsic drive to interact with the block early in training, the ICM agent's behaviour collapsed towards the standard SAC+HER baseline. In fact, the added computational overhead and slight objective dilution introduced by the ICM module likely explain why it converged more slowly than the pure Learnt α baseline, ultimately proving that transition-

based curiosity is poorly suited for highly deterministic simulation environments compared to spatial novelty-based approaches like RND.

7.4 Hardware Viability and Deployment Readiness

While success rate and sample efficiency are primary indicators of an algorithm's theoretical performance, real-world robotic deployment demands a critical additional layer of scrutiny: trajectory safety and hardware viability. To quantify this, the Mean Absolute Jerk was recorded for each agent at incremental stages of training. However, quantitative metrics alone do not capture the full behavioural profile of the agents. Qualitative observations of the fully trained policies revealed stark differences in post-task-completion behaviour, directly impacting their readiness for physical hardware.

The **Fixed α Baseline** exhibited the most conservative physical behaviour. Upon successfully placing the block in the target radius, the agent consistently halted movement, maintaining a static pose for the remainder of the episode.

The **Learnt α Baseline** demonstrated similar task-oriented behaviour, but with an observable degradation in post-completion stability. While the arm successfully secured the block, it occasionally exhibited stochastic joint oscillations, or "jitter," at the end of the trajectory. Although the temperature parameter plateaued at a low value (0.0035), the non-zero entropy still introduces continuous noise into the action space, which manifests physically as actuator twitching.

This presents an intriguing behavioural paradox: the agent mathematically mandated to maintain a higher exploratory drive (the fixed α baseline) exhibited the most physically conservative behaviour after task completion, whereas the baseline agent with a diminished exploratory drive (learnt α) demonstrated much more erratic stabilisation.

The most profound behavioural deviations, however, were observed in the **Intrinsically Motivated Agents (RND and ICM)**. While both achieved high success rates and rapid task execution, qualitative observations revealed highly erratic, non-functional trajectories after the primary task was completed. Once the block was placed at the target, both agents frequently twisted their joints, swayed the arm across the workspace, and executed seemingly random, jolty, and unnatural spatial movements until the episode timed out. Visually, the ICM agent appeared just as jittery as the RND agent during this post-completion phase.

This erratic behaviour is a direct consequence of the intrinsic motivation architectures. Because the environment operates on fixed-length episodes, finishing the task early leaves the agents with surplus timesteps. The exploration modules inadvertently encourage "reward farming." The agents learn to secure the extrinsic task reward first, and then utilise the remaining time to maximise their intrinsic reward yields—RND by seeking spatial novelty in unvisited Cartesian coordinates, and ICM by inducing jolty, chaotic joint movements that its forward model struggles to perfectly predict.

Figure 8 illustrates the evolution of Mean Absolute Jerk across the training process, quantitatively confirming these visual observations. The fixed α

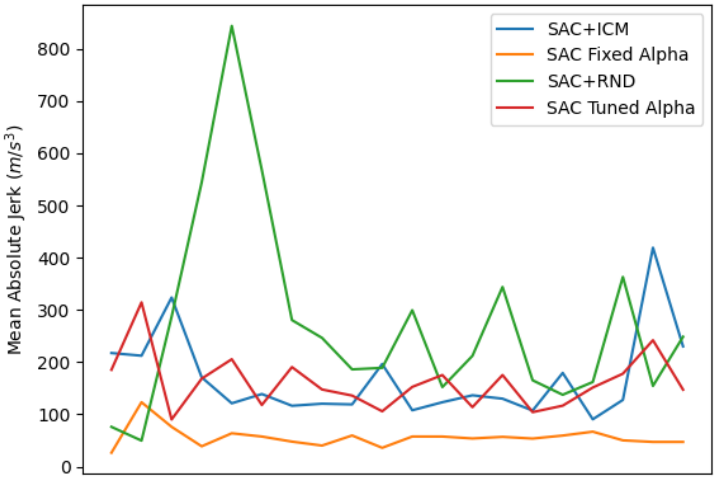

Figure 8: Mean Absolute Jerk at each stage of training

baseline demonstrates minimal motion jitter, converging to approximately 40 m/s^3. This low Mean Absolute Jerk suggests it is the superior algorithmic choice regarding physical hardware viability. The learnt α baseline, reflecting its continuous active stabilisation, exhibits moderate variance throughout before settling at a Mean Absolute Jerk of approximately 150 m/s^3.

However, the physical toll of intrinsic reward farming is starkly reflected in the RND and ICM agents, both of which exhibit high variance throughout training before converging to a severe Mean Absolute Jerk of approximately 250 m/s^3, with peaks, particularly RND, exceeding $800m/s^3$. This level of jerk is highly aggressive for physical deployment; equating to a change in acceleration of over $25g$ per second, these unnatural, high-frequency movements would induce catastrophic mechanical strain and rapid actuator wear on a physical 7-DOF manipulator. These findings highlight a critical trade-off: while intrinsic motivation can accelerate simulated task convergence, without explicit jerk penalties or reward shaping, it generates policies that are fundamentally unsafe for direct zero-shot deployment on physical hardware.

8 Discussion

This study demonstrates that intrinsic motivation reliably accelerates sparse-reward convergence in robotic manipulation, but at a cost that conventional benchmarking metrics fail to detect. Taken together, the success rate, sample efficiency, and Mean Absolute Jerk analyses reveal a fundamental tension: the exploration mechanisms most responsible for sample efficiency gains are the same mechanisms that render trained policies physically unsafe. This trade-off has critical implications for the sim-to-real deployment of intrinsically moti-

vated agents and motivates a broader reconsideration of how such algorithms are evaluated.

8.1 Entropy Scheduling as a Mechanistic Factor

The most immediate finding is that entropy scheduling alone accounts for a substantial portion of the performance gap between the two baselines. Haarnoja et al. [2] introduced automatic temperature tuning in SAC precisely to resolve the conflict between early-training exploration and late-training policy precision, arguing that a fixed coefficient cannot simultaneously satisfy both requirements. The results presented here provide direct empirical validation of this claim in the context of dexterous manipulation. The learnt α agent's rapid decay from 1.0 to a near-deterministic plateau of 0.0035 reflects the fundamental nature of the pick-and-place task: broad stochasticity is necessary to initially locate the block and target, but the strict 0.05m success radius demands that the policy subsequently collapse into precise, repeatable motor commands. The fixed α baseline's inability to make this transition is therefore not an architectural deficiency but a principled consequence of maintaining entropy in a task that requires eventual determinism.

This leads to a notable behavioural paradox that warrants explicit attention: the agent mathematically mandated to maintain the *higher* exploratory drive exhibited the most physically conservative post-completion behaviour, consistently halting at the goal state. The learnt α agent, despite its superior success rate, exhibited stochastic joint oscillations after task completion—a direct physical manifestation of the residual non-zero entropy (0.0035). This **entropy-stiffness paradox** reveals that in rigid-body manipulation tasks, a policy trained with sustained stochasticity may inadvertently learn to suppress motion as a survival mechanism against its own mandated noise, while a more confident policy continuously attempts micro-corrections that manifest as actuator twitching. This observation extends Haarnoja et al.'s theoretical motivation for temperature tuning into a physical, hardware-relevant consequence that is not captured by task reward alone.

8.2 Intrinsic Motivation: A Natural Experiment in Curiosity Design

The divergence between RND and ICM performance constitutes a natural experiment that distinguishes state-novelty from transition-novelty as exploration signals, with the determinism of the MuJoCo physics engine providing the isolating condition. Burda et al. [3] introduced RND specifically to overcome the non-stationarity and computational overhead of learned curiosity models, arguing that a fixed random network provides a stable and scalable novelty signal. The results here confirm this advantage empirically: the large initial spike in RND's intrinsic reward drove rapid Cartesian workspace coverage, providing the necessary early-training impetus to overcome the sparse reward plateau and yielding convergence at just 1.3 million timesteps.

ICM's failure to replicate this is mechanistically grounded in the design of its forward dynamics model. Pathak et al. [4] acknowledge that transition-based curiosity can be undermined by environmental stochasticity—the so-called "noisy TV problem," where unpredictable but task-irrelevant stimuli hijack the curiosity signal. The findings presented here reveal the *inverse* failure mode: an *overly* predictable environment kills the curiosity signal entirely. Because MuJoCo's rigid-body kinematics are deterministic, the ICM forward model rapidly saturates its prediction accuracy, producing a persistently low intrinsic reward regardless of whether the agent visits novel regions of state space. Moving the arm to an unvisited area generates no bonus because the *transition* was entirely predictable, even if the destination was novel. This represents a meaningful extension of the failure modes described by Pathak et al., identifying deterministic simulation as a structural condition under which transition-based curiosity is fundamentally mismatched with the exploration problem.

Crucially, this finding should not be interpreted as a blanket indictment of ICM for robotics. Rather, it implies that transition-based curiosity may recover its utility in environments with genuine stochasticity—for example, tasks involving perception noise, dynamic obstacles, or contact-rich manipulation where rigid-body dynamics are no longer perfectly predictable. The choice of exploration mechanism must therefore be matched to the structural properties of the simulation environment, not applied generically.

8.3 Hardware Viability and the Hidden Cost of Intrinsic Reward

The Mean Absolute Jerk analysis introduces a dimension of evaluation that purely reward-focused benchmarks systematically obscure. The reinforcement learning benchmarking literature has historically evaluated continuous control policies almost exclusively on task reward and sample efficiency. Mysore et al. [17] have warned that policies optimised under these criteria frequently exhibit high-frequency control signals that are incompatible with physical actuator constraints, but empirical evidence of this consequence in the context of intrinsic motivation specifically has remained sparse. The Mean Absolute Jerk results presented here provide precisely this evidence.

While RND and ICM both achieve competitive success rates (0.93 and 0.90 respectively), both converge to a Mean Absolute Jerk of approximately 250 m/s^3—equivalent to over $25g$ of jerk per second on the end-effector. On a physical 7-DOF manipulator such as the Fetch robot, jerk at this magnitude would induce severe mechanical strain across the kinematic chain: repeated acceleration shocks of this magnitude accelerate gear wear in the joint actuators, excite structural resonances in the links, and risk destabilising any payload held by the gripper. By contrast, the fixed α baseline converges to approximately 40 m/s^3, a level plausibly compatible with physical deployment.

The causal chain underlying this finding is direct and important to state explicitly: the sparse reward structure requires the agent to reach the block before any learning signal is received; the intrinsic bonus accelerates this by incentivis-

ing workspace exploration; completing the task early leaves surplus timesteps in the fixed-length episode; the exploration module then incentivises "reward farming"—RND by seeking unvisited Cartesian coordinates, ICM by inducing chaotic joint movements that its forward model struggles to predict. The same mechanism that produces sample efficiency gains is therefore the mechanism that produces catastrophic jerk. This is not an incidental implementation flaw but a structural consequence of combining open-ended exploration bonuses with fixed-length episodes in a task that can be completed well before the episode terminates.

A further counterintuitive implication deserves emphasis: the safest policy in terms of hardware viability is the one with the *lowest* task performance. This is not coincidental. The fixed α agent's mandated stochasticity inadvertently discourages the directed, high-amplitude post-completion movements that the exploration modules reward, producing conservative behaviour as an emergent property. This finding suggests that simulated success rate is a poor proxy for deployment readiness, and that hardware viability metrics such as Mean Absolute Jerk must be treated as co-equal evaluation criteria rather than secondary concerns.

8.4 Methodological Limitations

Three methodological constraints temper the conclusions of this study, though none fatally undermines the relative comparisons drawn between agents.

The most consequential limitation is the use of fixed-length episodes. The reward farming behaviour observed in the RND and ICM agents is a direct artefact of this design: surplus timesteps provide the exploration modules with an unconstrained window to maximise intrinsic reward after the primary task is completed. Notably, however, this design choice inadvertently constitutes an experimental revelation rather than merely a flaw. The erratic post-completion behaviour observed here would manifest on physical hardware whenever a task is completed before the controller is disengaged—a scenario that is entirely plausible in real deployment. The results therefore expose a genuine safety risk in these algorithms that might otherwise have remained latent under an early-termination protocol. Future work should implement an early-termination condition—halting the episode once the block is stabilised within the success radius for a fixed number of consecutive frames—and measure whether this eliminates the Mean Absolute Jerk penalty while preserving the sample efficiency advantage.

The second constraint concerns the Mean Absolute Jerk metric itself. Because the *Fetch-Pick-And-Place-v4* environment wrapper abstracts away direct actuator states, the Mean Absolute Jerk was derived from end-effector Cartesian velocities rather than raw joint-level angular velocities or torques. This means the metric captures task-space smoothness—how fluidly the policy directs the gripper through three-dimensional space—rather than joint-level mechanical wear directly. Critically, however, because all four agents were evaluated under identical environment conditions, the Mean Absolute Jerk remains a valid

relative measure of differential smoothness across agents. The absolute values cannot be mapped directly to actuator torques without joint-level instrumentation, but the ordering and magnitude ratios between agents are mechanistically meaningful. Future work employing joint-torque or angular jerk metrics would provide a more direct assessment of physical wear and allow the absolute Mean Absolute Jerk thresholds reported here to be translated into hardware-specific safety bounds.

Finally, the stochastic block initialisation of the environment occasionally places the block within the target radius at episode start, granting an immediate spurious success. While this occurs infrequently (approximately once every 100–200 episodes) and does not materially distort the aggregate metrics over 4.75 million timesteps, it introduces a small variance into sample efficiency measurements that is not reflective of genuine policy learning.

8.5 Future Directions

The findings of this study motivate four concrete directions for future investigation. First, the early-termination correction described above should be implemented and tested to determine whether it preserves the sample efficiency advantage of RND and ICM while eliminating the reward farming behaviour responsible for elevated Mean Absolute Jerk. Second, the ICM/determinism result motivates testing transition-based curiosity in environments with genuine stochasticity—contact-rich manipulation, tasks with perception noise, or environments with dynamic obstacles—to identify the conditions under which ICM's exploration signal recovers utility relative to RND. Third, future studies should instrument the simulation at the joint level to obtain torque-rate and angular jerk metrics, enabling a direct validation of whether end-effector Mean Absolute Jerk is a reliable proxy for actuator-level mechanical wear. Fourth, and most broadly, the tension between intrinsic motivation and physical smoothness motivates the investigation of architectures that explicitly incorporate jerk penalties or actuator-aware reward shaping alongside exploration bonuses, with the aim of recovering the sample efficiency advantages of RND and ICM without sacrificing deployment safety.

8.6 Broader Implications

The standard benchmarking vocabulary of deep reinforcement learning—success rate, mean episode reward, convergence timesteps—is insufficient for evaluating policies intended for physical deployment. This study, by introducing Mean Absolute Jerk as a co-equal evaluation criterion alongside task performance metrics, demonstrates that algorithms which appear optimal by conventional measures can be fundamentally incompatible with hardware constraints. The finding that intrinsic motivation accelerates simulated convergence while simultaneously producing policies that would cause catastrophic mechanical strain on physical hardware represents a critical and underappreciated blind spot in the current intrinsic motivation literature. Bridging this gap requires not only

methodological extensions—joint-level metrics, early-termination protocols, explicit smoothness constraints—but a cultural shift in how the field defines success: one that treats physical viability as a first-class objective rather than an afterthought to simulated performance.

9 Conclusion

This study set out to determine whether intrinsic motivation meaningfully improves upon a SAC+HER baseline in sparse-reward robotic manipulation, and whether the resulting policies are viable for physical deployment. The answer to both questions is clear. RND accelerated convergence substantially, confirming the value of state-novelty exploration in this task class; ICM was rendered ineffective by the deterministic MuJoCo environment, identifying a structural mismatch between transition-based curiosity and predictable simulation physics. Critically, however, the Mean Absolute Jerk analysis reveals that both intrinsically motivated agents converge to trajectory profiles that would be catastrophic on physical hardware, while the weakest-performing baseline produced the only policy compatible with zero-shot deployment.

The central contribution of this work is therefore methodological as much as empirical: simulated success rate is a poor proxy for deployment readiness. Hardware viability metrics must be treated as first-class evaluation criteria alongside task performance. Future work incorporating early-termination, joint-level instrumentation, and explicit smoothness constraints may recover the sample efficiency advantages of intrinsic motivation without sacrificing physical safety.

10 References

References

[1] Timothy P. Lillicrap et al. "Continuous control with deep reinforcement learning". In: *ICLR (Poster)*. 2016. URL: http://arxiv.org/abs/1509.02971.

[2] Tuomas Haarnoja et al. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". In: *International Conference on Machine Learning (ICML)* (2018).

[3] Yuri Burda et al. "Exploration by Random Network Distillation". In: *CoRR* abs/1810.12894 (2018). arXiv: 1810.12894. URL: http://arxiv.org/abs/1810.12894.

[4] Deepak Pathak et al. "Curiosity-driven exploration by self-supervised prediction". In: *International conference on machine learning*. PMLR. 2017, pp. 2778–2787.

[5] Matthias Plappert et al. *Multi-Goal Reinforcement Learning: Challenging Robotics Environments and Request for Research*. 2018. eprint: arXiv:1802.09464.

[6] John Schulman et al. "Proximal policy optimization algorithms". In: *arXiv preprint arXiv:1707.06347* (2017).

[7] Scott Fujimoto, Herke Hoof, and David Meger. "Addressing function approximation error in actor-critic methods". In: *International conference on machine learning*. PMLR. 2018, pp. 1587–1596.

[8] Marcin Andrychowicz et al. "Hindsight experience replay". In: *Advances in neural information processing systems* 30 (2017).

[9] Diederik P. Kingma and Max Welling. "Auto-Encoding Variational Bayes". In: *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*. Ed. by Yoshua Bengio and Yann LeCun. 2014. URL: http://arxiv.org/abs/1312.6114.

[10] GeeksforGeeks. *Soft Actor-Critic Reinforcement Learning Algorithm — diagram*. n.d. URL: https://www.geeksforgeeks.org/deep-learning/soft-actor-critic-reinforcement-learning-algorithm/ (visited on 12/05/2025).

[11] Tuomas Haarnoja et al. "Soft Actor-Critic Algorithms and Applications". In: *arXiv:1812.05905 [cs, stat]* (Jan. 2019). URL: https://arxiv.org/abs/1812.05905.

[12] Marc Bellemare et al. "Unifying count-based exploration and intrinsic motivation". In: *Advances in neural information processing systems* 29 (2016).

[13] Arthur Aubret, Laëtitia Matignon, and Salima Hassas. "A survey on intrinsic motivation in reinforcement learning". In: *CoRR* abs/1908.06976 (2019). arXiv: 1908.06976. URL: http://arxiv.org/abs/1908.06976.

[14] Giacomo Rosa and Nir Lipovetzky. "Count-Based Novelty Exploration in Classical Planning". In: *ECAI 2024 - 27th European Conference on Artificial Intelligence, 19-24 October 2024, Santiago de Compostela, Spain - Including 13th Conference on Prestigious Applications of Intelligent Systems (PAIS 2024)*. Ed. by Ulle Endriss et al. Frontiers in Artificial Intelligence and Applications. IOS Press, 2024, pp. 4181–4189. DOI: 10.3233/FAIA240990. URL: https://doi.org/10.3233/FAIA240990.

[15] Shakir Mohamed and Danilo Jimenez Rezende. "Variational information maximisation for intrinsically motivated reinforcement learning". In: *Advances in neural information processing systems* 28 (2015).

[16] Seyed Adel Alizadeh Kolagar, Mehdi Heydari Shahna, and Jouni Mattila. "Combining Deep Reinforcement Learning with a Jerk-Bounded Trajectory Generator for Kinematically Constrained Motion Planning*". In: *2025 European Control Conference (ECC)*. IEEE, June 2025, pp. 2507–2514. DOI: 10.23919/ecc65951.2025.11187262. URL: http://dx.doi.org/10.23919/ECC65951.2025.11187262.

[17] Siddharth Mysore et al. "Regularizing action policies for smooth control with reinforcement learning". In: *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. 2021, pp. 1810–1816.

[18] Shengyi Huang et al. "CleanRL: High-quality Single-file Implementations of Deep Reinforcement Learning Algorithms". In: *CoRR* abs/2111.08819 (2021). arXiv: 2111.08819. URL: https://arxiv.org/abs/2111.08819.

[19] Barry Payne Welford. "Note on a method for calculating corrected sums of squares and products". In: *Technometrics* 4.3 (1962), pp. 419–420.

[20] Andrew M. Saxe, James L. McClelland, and Surya Ganguli. "Exact solutions to the nonlinear dynamics of learning in deep linear neural networks". In: *2nd International Conference on Learning Representations (ICLR)*. 2014.

11 Appendix

Table 3: Neural network architectures for the SAC agent and intrinsic motivation modules.

Network	**Input Dim.**	**Hidden Layers**	**Output Dim.**	**Activation**	**Initialization**
SAC Actor	$\|s\| + \|g\|$	$[256, 256, 256]$	$\|a\|$ (Mean & Log Std)	ReLU	PyTorch Default
SAC Critic ($\times 2$)	$\|s\| + \|g\| + \|a\|$	$[256, 256, 256]$	1 (Q-Value)	ReLU	PyTorch Default
RND Target	$\|s\|$	$[512, 512, 512, 512]$	512	LeakyReLU	Orthogonal ($\sqrt{2}$)
RND Predictor	$\|s\|$	$[512, 512, 512, 512]$	512	LeakyReLU	Orthogonal ($\sqrt{2}$)
ICM Forward	$\|s\| + \|a\|$	$[256, 256]$	$\|s\|$	ReLU	Orthogonal ($\sqrt{2}$)

Note: $|s|$, $|g|$, and $|a|$ denote the dimensions of the state, goal, and action spaces, respectively.

Table 4: Key hyperparameters used during training and optimization.

Hyperparameter	**Symbol**	**Value**
Learning Rate (All Networks)	lr	3×10^{-4}
Discount Factor	γ	0.98
Target Smoothing Coefficient	τ	0.05
Intrinsic Reward Scale (RND)	β	0.5
Intrinsic Reward Scale (ICM)	β	0.2
Fixed Entropy Coefficient	α	0.2
Target Entropy (Tuned α)	$\bar{\mathcal{H}}$	$-\dim(\mathcal{A})$
HER Future Relabeling Ratio	k	4
Optimiser	-	Adam

Note: $\mathcal{A}$ denotes the action space.